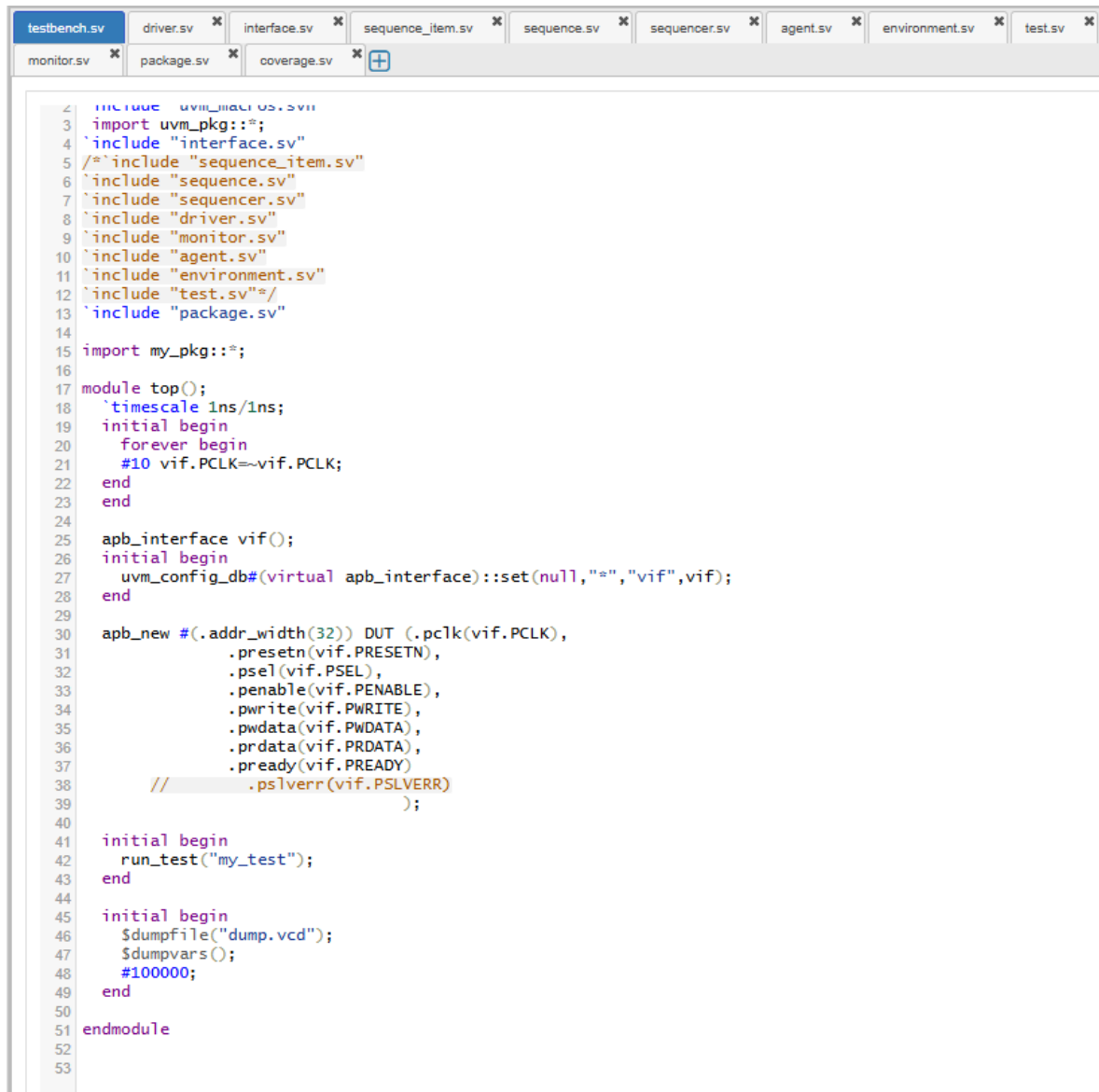


UVM based testbench using APB protocol



```
1  testbench.sv x driver.sv x interface.sv x sequence_item.sv x sequence.sv x sequencer.sv x agent.sv x environment.sv x test.sv x
2  monitor.sv x package.sv x coverage.sv x +
3
4  1 include uvm_macros.svh
5  2 import uvm_pkg::*;
6  3 `include "interface.sv"
7  4 /*`include "sequence_item.sv"
8  5 `include "sequence.sv"
9  6 `include "sequencer.sv"
10 7 `include "driver.sv"
11 8 `include "monitor.sv"
12 9 `include "agent.sv"
13 10 `include "environment.sv"
14 11 `include "test.sv"*/
15 12 `include "package.sv"
16 13
17 14 import my_pkg::*;
18 15
19 16 module top();
20 17     `timescale 1ns/1ns;
21 18     initial begin
22 19         forever begin
23 20             #10 vif.PCLK=~vif.PCLK;
24 21         end
25 22     end
26 23
27 24 apb_interface vif();
28 25 initial begin
29 26     uvm_config_db#(virtual apb_interface)::set(null, "*", "vif", vif);
30 27 end
31 28
32 29 apb_new #(.addr_width(32)) DUT (.pclk(vif.PCLK),
33 30     .presetn(vif.PRESETN),
34 31     .psel(vif.PSEL),
35 32     .penable(vif.PENABLE),
36 33     .pwrite(vif.PWRITE),
37 34     .pwrite(vif.PWRITE),
38 35     .pwrite(vif.PWRITE),
39 36     .prdata(vif.PRDATA),
40 37     .prdata(vif.PRDATA),
41 38     .pready(vif.PREADY)
42 39     // .pslverr(vif.PSLVERR)
43 40 );
44 41
45 42 initial begin
46 43     run_test("my_test");
47 44 end
48 45
49 46 initial begin
50 47     $dumpfile("dump.vcd");
51 48     $dumpvars();
52 49     #100000;
53 50 end
54 51 endmodule
55 52
56 53
```

testbench.sv

driver.sv

interface.sv

sequence_item.sv

sequence.sv

sequencer.sv

agent.sv

environment.sv

test.sv

monitor.sv

package.sv

coverage.sv

```

1 class driver extends uvm_driver#(seq_item);
2   `uvm_component_utils(driver)
3   function new(string name="driver",uvm_component parent=null);
4     super.new(name,parent);
5     req=new();
6   endfunction
7
8   seq_item req;
9   virtual apb_interface vif;
10
11   function void build_phase(uvm_phase phase);
12     if(!uvm_config_db#(virtual apb_interface)::get(this,"", "vif",vif))
13       `uvm_error("vif interface","vif is not getting");
14     req=seq_item::type_id::create("req");
15   endfunction
16
17   task run_phase(uvm_phase phase);
18     forever begin
19       seq_item_port.get_next_item(req);
20       drive_item();
21       $display("123456");
22       seq_item_port.item_done();
23     end
24   endtask
25
26   task drive_item();
27     begin
28       if(req.packet_h==2)
29         begin
30           vif.PSEL<=1'b0;
31           vif.PENABLE<=1'b0;
32           vif.PWRITE<=1'b0;
33           vif.PADDR<=32'b0;
34           vif.PWDATA<=32'b0;
35           vif.PRDATA<=32'b0;
36         end
37       else
38         begin
39           @(posedge vif.PCLK)
40             vif.PSEL<=1'b0;
41             vif.PENABLE<=1'b0;
42             vif.PWRITE<=1'b0;
43             vif.PADDR<=32'b0;
44             vif.PWDATA<=32'b0;
45             vif.PRDATA<=32'b0;
46           @(posedge vif.PCLK)
47             vif.PSEL<=1'b1;
48             vif.PENABLE<=1'b0;
49             vif.PWRITE<=req.PWRITE;
50             vif.PADDR<=req.PADDR;
51             if(req.PWRITE)
52               vif.PWDATA<=req.PWDATA;
53             @(posedge vif.PCLK)
54               vif.PRDATA<=req.PRDATA;
55             vif.PADDR<=req.PADDR;
56             if(req.PWRITE)
57               vif.PWDATA<=req.PWDATA;
58             @(posedge vif.PCLK)
59               vif.PSEL<=1'b1;
60               vif.PENABLE<=1'b1;
61               wait(vif.PREADY);
62               if(vif.PWRITE)
63                 @(posedge vif.PCLK)
64                   if(vif.PSLVERR)
65                     `uvm_info("Transaction","error",UVM_NONE)
66                   else
67                     `uvm_info("No transaction","error",UVM_NONE)
68                 @(posedge vif.PCLK)
69                   vif.PSEL<=1'b0;
70                   vif.PENABLE<=1'b0;
71                 end
72             end
73         end
74     end
75   endtask
76 endclass

```

testbench.sv	driver.sv	interface.sv	sequence_item.sv	sequence.sv	sequencer.sv	agent.sv	environment.sv	test.sv
monitor.sv	package.sv	coverage.sv	+					

```

1 interface apb_interface();
2   bit PCLK=0;
3   logic PRESETN;
4   logic PSEL;
5   logic PENABLE;
6   logic PWRITE;
7   logic [31:0] PADDR;
8   logic [31:0] PWDATA;
9   logic [31:0] PRDATA;
10  logic PREADY;
11  logic PSLVERR;
12  logic PWAKEUP;
13  logic [3:0]PSTRB;
14
15  clocking driver_cb@(posedge PCLK);
16    default input #1 output #1;
17    output PSEL;
18    output PENABLE;
19    output PWRITE;
20    output PWDATA;
21    output PADDR;
22    output PWAKEUP;
23    output PSTRB;
24    input PRDATA;
25    input PSLVERR;
26    input PREADY;
27  endclocking
28
29  clocking monitor_cb@(posedge PCLK);
30    default input #1 output #1;
31    input PSEL;
32    input PENABLE;
33    input PWRITE;
34    input PWDATA;
35    input PADDR;
36    input PWAKEUP;
37    input PSTRB;
38    input PRDATA;
39    input PSLVERR;
40    input PREADY;
41  endclocking
42 endinterface

```

testbench.sv	driver.sv	interface.sv	sequence_item.sv	sequence.sv	sequencer.sv	agent.sv	environment.sv	test.sv
monitor.sv	package.sv	coverage.sv	+					

```

1 class seq_item extends uvm_sequence_item;
2
3   `uvm_object_utils(seq_item)
4   rand bit[31:0] PADDR;
5   rand bit[31:0] PWDATA;
6   rand bit PWRITE;
7   bit[31:0] PRDATA;
8   bit[3:0] PSTRB;
9   bit PRESETN;
10  bit PSEL;
11  bit PENABLE;
12  bit PREADY;
13  bit PWAKEUP;
14  bit PSLVERR;
15
16  //typedef enum bit [1:0] {write,read,reset} packet;
17  rand packet packet_h;
18  //typedef enum bit [1:0]{idle,setup,access} phase;
19  randc phase phase_h;
20
21  constraint c {if(packet_h==write)
22  {
23      soft PADDR inside{[1:15]};
24      PWDATA inside{[1:15]};
25      PWRITE==1;
26      PRESETN == 1'b1;
27  }
28  else if(packet_h==read)
29  {
30      soft PADDR inside{[1:15]};
31      PWRITE==0;
32      PRESETN == 1'b1;
33  }
34  else {PRESETN == 1'b0; PADDR == 32'h00000000; PWRITE ==1'b0; PWDATA==32'h00000000;}
35  }
36      function new(string name="seq_item");
37          super.new(name);
38      endfunction
39
40 endclass

```

testbench.sv	driver.sv	interface.sv	sequence_item.sv	sequence.sv	sequencer.sv	agent.sv	environment.sv	test.sv
monitor.sv	package.sv	coverage.sv	+					

```

1 class seq_write extends uvm_sequence#(seq_item);
2   `uvm_object_utils(seq_write)
3   int addr;
4   function new(string name="seq_write");
5       super.new();
6   endfunction
7
8   task body();
9       begin
10          req=seq_item::type_id::create("req");
11          start_item(req);
12          assert(req.randomize with{req.packet_h==reset;});
13          $display("randomization done");
14          $display("randomizedvalue=%0d",req);
15          finish_item(req);
16          repeat(10)
17          begin
18              req=seq_item::type_id::create("req");
19              start_item(req);
20              assert(req.randomize with{req.packet_h==write;});
21              $display("randomization done");
22              $display("randomizedvalue=%0d",req);
23              addr = req.PADDR;
24              finish_item(req);
25              req=seq_item::type_id::create("req");
26              start_item(req);
27              assert(req.randomize with{req.packet_h==read;
28                  req.PADDR == addr;});
29              $display("randomization done");
30              $display("randomizedvalue=%0d",req);
31              finish_item(req);
32          end
33      end
34  endtask
35 endclass

```

testbench.sv	driver.sv ✕	interface.sv ✕	sequence_item.sv ✕	sequence.sv ✕	sequencer.sv ✕	agent.sv ✕	environment.sv ✕	test.sv ✕
monitor.sv ✕	package.sv ✕	coverage.sv ✕	+					

```

1 class sequencer extends uvm_sequencer#(seq_item);
2
3   `uvm_component_utils(sequencer)
4
5   function new(string name="sequencer",uvm_component parent);
6     super.new(name,parent);
7   endfunction
8
9   function void build_phase(uvm_phase phase);
10     super.build_phase(phase);
11   endfunction
12
13 endclass

```

testbench.sv	driver.sv ✕	interface.sv ✕	sequence_item.sv ✕	sequence.sv ✕	sequencer.sv ✕	agent.sv ✕	environment.sv ✕	test.sv ✕
monitor.sv ✕	package.sv ✕	coverage.sv ✕	+					

```

1 class agent extends uvm_agent;
2
3   `uvm_component_utils(agent);
4
5   sequencer sequencer_h;
6   driver drv_h;
7   mon mon_h;
8
9   function new(string name="agent",uvm_component parent);
10     super.new(name,parent);
11   endfunction
12
13   function void build_phase(uvm_phase phase);
14     super.build_phase(phase);
15     sequencer_h=sequencer::type_id::create("sequencer_h",this);
16     drv_h=driver::type_id::create("drv_h",this);
17     mon_h=mon::type_id::create("mon_h",this);
18   endfunction
19
20   function void connect_phase(uvm_phase phase);
21     super.connect_phase(phase);
22     drv_h.seq_item_port.connect(sequencer_h.seq_item_export);
23   endfunction
24 endclass

```

testbench.sv	driver.sv ✕	interface.sv ✕	sequence_item.sv ✕	sequence.sv ✕	sequencer.sv ✕	agent.sv ✕	environment.sv ✕	test.sv ✕
monitor.sv ✕	package.sv ✕	coverage.sv ✕	+					

```

1 class env extends uvm_env;
2
3   `uvm_component_utils(env);
4   agent agt_h;
5   //scoreboard scbd_h;
6   coverage cov_h;
7
8   function new(string name="env",uvm_component parent);
9     super.new(name,parent);
10   endfunction
11
12   function void build_phase(uvm_phase phase);
13     super.build_phase(phase);
14     agt_h=agent::type_id::create("agt_h",this);
15     //scbd_h=scoreboard::type_id::create("scbd_h",this);
16     cov_h=coverage::type_id::create("cov_h",this);
17   endfunction
18
19   function void connect_phase(uvm_phase phase);
20     super.connect_phase(phase);
21     //agt_h.mon_h.m_analysis_port.connect(scoreboard_h.sb_export.analysis_export);
22   endfunction
23 endclass

```

testbench.sv x driver.sv x interface.sv x sequence_item.sv x sequence.sv x sequencer.sv x agent.sv x environment.sv x test.sv x

monitor.sv x package.sv x coverage.sv x +

```
1 class my_test extends uvm_test;
2
3   `uvm_component_utils(my_test);
4   env env_h;
5   seq_write write_h;
6   virtual apb_interface vif;
7
8   function new(string name="my_test",uvm_component parent);
9     super.new(name,parent);
10  endfunction
11
12  function void build_phase(uvm_phase phase);
13    super.build_phase(phase);
14    env_h=env::type_id::create("env_h",this);
15    write_h=seq_write::type_id::create("write_h",this);
16  endfunction
17
18  function void end_of_elaboration_phase(uvm_phase phase);
19    super.end_of_elaboration_phase(phase);
20    uvm_top.print_topology();
21  endfunction
22
23  task run_phase(uvm_phase phase);
24    phase.raise_objection(this);
25    write_h.start(env_h.agt_h.sequencer_h);
26    #200
27    phase.drop_objection(this);
28  endtask
29
30 endclass
```

```
testbench.sv x driver.sv x interface.sv x sequence_item.sv x sequence.sv x sequencer.sv x agent.sv x environment.sv x test.sv x
monitor.sv x package.sv x coverage.sv x +

1 class mon extends uvm_monitor;
2
3   `uvm_component_utils(mon)
4
5   uvm_analysis_port #(seq_item) m_analysis_port;
6   int start;
7   int stop;
8   int count;
9
10  seq_item req;
11  virtual apb_interface vif;
12
13  function new(string name="mon",uvm_component parent);
14    super.new(name,parent);
15    m_analysis_port=new("m_analysis_port",this);
16  endfunction
17
18  function void build_phase(uvm_phase phase);
19    super.build_phase(phase);
20    req=seq_item::type_id::create("req");
21    if(!uvm_config_db #(virtual apb_interface)::get(this,"","vif",vif))
22      `uvm_error("vif interface","interface is not getting")
23  endfunction
24
25  task start_stop();
26    forever begin
27      @(posedge vif.PCLK)
28        if(vif.PSEL==1)
29          start=1;
30        else if(start==1)
31          stop=1;
32      `uvm_info("The start and ", "stop thread has been executed",UVM_NONE);
33    end
34  endtask
35
36  task capture_data();
37    forever begin
38      @(posedge vif.PCLK)
39        case({vif.PSEL,vif.PENABLE})
40          2'b00:
41            begin
42              req.phase_h=0;
43              req.PSEL=vif.PSEL;
44              req.PWRITE=vif.PWRITE;
45              req.PENABLE=vif.PENABLE;
46              req.PADDR=vif.PADDR;
47              req.PRDATA=vif.PRDATA;
48              req.PWDATA=vif.PWDATA;
49              m_analysis_port.write(req);
50            end
51          2'b10:
52            begin
53
```

```

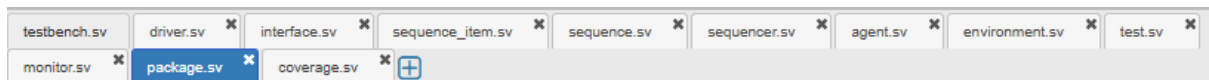
48         m_analysis_port.write(req);
49     end
50
51
52     2'b10:
53     begin
54         req.phase_h=1;
55         req.PADDR=vif.PADDR;
56         req.PWRITE=vif.PWRITE;
57         @(posedge vif.PCLK)
58         req.PRDATA=vif.PRDATA;
59         req.PWDATA=vif.PWDATA;
60         m_analysis_port.write(req);
61     end
62
63     2'b11:
64     begin
65         req.phase_h=access;
66         req.PENABLE=vif.PENABLE;
67         if(vif.PWRITE)
68         begin
69             req.PWDATA=vif.PWDATA;
70             @(posedge vif.PCLK)
71             req.PSLVERR=vif.PSLVERR;
72         end
73         else
74         begin
75             req.PRDATA=vif.PRDATA;
76             req.PSLVERR=vif.PSLVERR;
77         end
78         m_analysis_port.write(req);
79     end
80 endcase
81
82     error_checking();//THREAD 3
83 end
84 endtask
85
86 function void error_checking();
87 begin
88     //forever
89     begin
90         //@(posedge vif.PCLK)
91         if(!vif.PSEL && vif.PENABLE)
92             `uvm_error("Checker","it has not entered the setup phase")
93
94
95         if(vif.PSLVERR)
96             `uvm_error("Checker","PSLVERR is found")
97
98
99         if(req.phase_h == access && !vif.PREADY)
100             count++;
101     else if(count>16)

```

```

95         if(vif.PSLVERR)
96             `uvm_error("Checker","PSLVERR is found")
97
98
99         if(req.phase_h == access && !vif.PREADY)
100             count++;
101         else if(count>16)
102             `uvm_error("Checker","PREADY can't be low for 16 cycles")
103
104
105
106         if(req.phase_h== idle && (req.PSEL!=0 || req.PWRITE!=0 || req.PENABLE!=0 ||
req.PADDR!=0 || /*req.PRDATA!=0 ||*/ req.PWDATA!=0))
107             `uvm_error(get_type_name(),"There is an error in the IDLE phase")
108
109         else if(req.phase_h== setup && (req.PSEL==0 || req.PWRITE==1'bx || req.PENABLE!=0
|| req.PADDR==1'bx || req.PRDATA==1'bx || req.PWDATA==1'bx))
110             `uvm_error("Checker","There is an error in the SETUP phase")
111
112         else if(req.phase_h== access && (req.PSEL==0 || req.PWRITE==1'bx || req.PENABLE==0
|| req.PADDR==1'bx || req.PRDATA==1'bx || req.PWDATA==1'bx))
113             `uvm_error("Checker","There is an error in the ACCESS phase")
114
115         end
116     end
117 endfunction
118
119 task run_phase(uvm_phase phase);
120     super.run_phase(phase);
121     fork
122         start_stop();//THREAD 1
123         //begin
124         capture_data();//THREAD 2
125
126         //error_checking();//THREAD 3
127         //end
128     join
129 endtask
130 endclass

```



```

1 |
2 package my_pkg;
3 `include "uvm_macros.svh"
4 import uvm_pkg::*;
5 typedef enum bit [1:0] {write,read,reset} packet;
6 typedef enum bit [1:0] {idle,setup,access} phase;
7
8 `include "sequence_item.sv"
9 `include "sequence.sv"
10 `include "sequencer.sv"
11 `include "driver.sv"
12 `include "monitor.sv"
13 `include "agent.sv"
14 `include "coverage.sv"
15 `include "environment.sv"
16 `include "test.sv"
17
18 endpackage
19
20

```

testbench.sv	driver.sv ✕	interface.sv ✕	sequence_item.sv ✕	sequence.sv ✕	sequencer.sv ✕	agent.sv ✕	environment.sv ✕	test.sv ✕
monitor.sv ✕	package.sv ✕	coverage.sv ✕	+					

```

1 class coverage extends uvm_subscriber#(seq_item);
2
3   `uvm_component_utils(coverage)
4   seq_item req;
5
6   covergroup cov;
7   option.per_instance=1;
8   coverpoint req.PADDR {bins b_addr={0:3}};
9   coverpoint req.PWDATA {bins b_pwdata={0:31}};
10  coverpoint req.PRDATA {bins b_prdata={0:31}};
11  coverpoint req.PWRITE {bins write={1};
12                               bins read={0};}
13
14  cross req.PADDR,req.PWDATA;
15  cross req.PADDR,req.PRDATA;
16 endgroup
17
18
19 function new(string name="coverage",uvm_component parent);
20   super.new(name,parent);
21   //cov=new();
22   req=new();
23   cov = new();
24   //c_export=new("c_export",this);
25   endfunction
26
27
28
29 function void build_phase(uvm_phase phase);
30   super.build_phase(phase);
31   endfunction
32
33
34 function void write(seq_item t);
35   //cg=new();
36   req=t;
37   cov.sample();
38   endfunction
39
40 endclass

```